In [1]:
```
import pandas as pd
```

In [2]:
```
import os

df = pd.read_csv(os.path.join(os.path.dirname(os.path.dirname(os.getcwd())), "Database", "interim_eda.csv"))
print(f"Loaded {len(df)} rows, {len(df.columns)} columns")
df.head()
```

```
Loaded 25000 rows, 20 columns
```

Out[2]:

	patient_id	age	gender	city	insurance_provider	chronic_flag	registration_date	visit_id	visit_date	department	visit_type	length_of_stay_hours	risk_score	doctor_id	bill_id	billed_amou
0	2964	25	F	Chennai	HealthPlus	1	2025-07-04	3	2025-07-13	ICU	ER	34.36	Low	153	3	5038.
1	4271	42	F	Mumbai	MediCareX	1	2025-10-31	669	2026-01-17	Neurology	ICU	12.74	Medium	138	669	21706.
2	1826	52	M	Pune	CareOne	1	2025-01-23	1264	2025-10-01	General	ICU	24.16	Low	123	1264	25918.
3	1224	52	M	Delhi	CareOne	1	2025-06-23	1326	2025-11-05	General	OPD	7.43	Low	134	1326	21173.
4	1318	39	M	Chennai	HealthPlus	1	2025-11-13	1455	2025-08-20	Cardiology	ER	17.72	High	136	1455	8649.

In [3]:
```
initial_shape = df.shape
print(f"Starting shape: {initial_shape}")

# Date Preparation
for col in ["registration_date", "visit_date", "billing_date"]:
    df[col] = pd.to_datetime(df[col])

reference_date = df["visit_date"].max()
print(f"Reference date (latest visit): {reference_date.date()}")

# --- Patient-Level Features ---
# Visit frequency per patient
visit_freq = df.groupby("patient_id")["visit_id"].nunique().reset_index(name="visit_frequency")
df = df.merge(visit_freq, on="patient_id", how="left")

# Average length of stay per patient
avg_los = df.groupby("patient_id")["length_of_stay_hours"].mean().reset_index(name="avg_length_of_stay")
df = df.merge(avg_los, on="patient_id", how="left")

# Days since registration (relative to reference date)
df["days_since_registration"] = (reference_date - df["registration_date"]).dt.days

# Total billed amount per patient
total_billed = df.groupby("patient_id")["billed_amount"].sum().reset_index(name="total_billed")
df = df.merge(total_billed, on="patient_id", how="left")

# Average payment days per patient
avg_pay = df.groupby("patient_id")["payment_days"].mean().reset_index(name="avg_payment_days")
df = df.merge(avg_pay, on="patient_id", how="left")
```

```python
# --- Provider/Doctor-Level Features ---
# Insurance provider rejection rate
provider_rej = df.groupby("insurance_provider")["claim_status"].apply(
    lambda x: (x == "Rejected").mean()
).reset_index(name="provider_rejection_rate")
df = df.merge(provider_rej, on="insurance_provider", how="left")

# Doctor rejection rate
doctor_rej = df.groupby("doctor_id")["claim_status"].apply(
    lambda x: (x == "Rejected").mean()
).reset_index(name="doctor_rejection_rate")
df = df.merge(doctor_rej, on="doctor_id", how="left")

# --- Time-Based Features ---
df["visit_month"] = df["visit_date"].dt.month
df["visit_day_of_week"] = df["visit_date"].dt.dayofweek
df["visit_quarter"] = df["visit_date"].dt.quarter
df["is_weekend"] = df["visit_day_of_week"].isin([5, 6]).astype(int)
df["days_between_reg_and_visit"] = (df["visit_date"] - df["registration_date"]).dt.days
df["billing_delay_days"] = (df["billing_date"] - df["visit_date"]).dt.days

# --- Validation ---
new_features = [
    "visit_frequency", "avg_length_of_stay", "days_since_registration",
    "total_billed", "avg_payment_days",
    "provider_rejection_rate", "doctor_rejection_rate",
    "visit_month", "visit_day_of_week", "visit_quarter", "is_weekend",
    "days_between_reg_and_visit", "billing_delay_days"
]

print(f"\nFinal shape: {df.shape} (added {df.shape[1] - initial_shape[1]} new columns)")
assert df.shape[0] == initial_shape[0], "Row count changed — merge issue!"
assert df[new_features].isna().sum().sum() == 0, "NaN values found in new features!"
print("✓ No rows lost, no NaN values in new features\n")

print("--- New Feature Summary ---")
df[new_features].describe().round(3)
```

```
Starting shape: (25000, 20)
Reference date (latest visit): 2026-01-20

Final shape: (25000, 33) (added 13 new columns)
✓ No rows lost, no NaN values in new features

--- New Feature Summary ---
```

Out[3]:

	visit_frequency	avg_length_of_stay	days_since_registration	total_billed	avg_payment_days	provider_rejection_rate	doctor_rejection_rate	visit_month	visit_day_of_week	visit_quarter	is_v
count	25000.000	25000.000	25000.000	25000.000	25000.000	25000.000	25000.000	25000.000	25000.000	25000.000	25
mean	5.961	19.552	185.321	124538.751	13.036	0.152	0.152	6.520	3.000	2.508	
std	2.185	5.509	105.692	55549.315	3.324	0.003	0.022	3.458	2.001	1.116	
min	1.000	0.500	0.000	500.000	1.000	0.149	0.106	1.000	0.000	1.000	
25%	4.000	15.935	93.000	84533.490	10.889	0.149	0.137	4.000	1.000	2.000	
50%	6.000	19.200	185.000	119562.600	12.845	0.150	0.151	7.000	3.000	3.000	
75%	7.000	22.838	279.000	158161.700	15.000	0.152	0.163	10.000	5.000	4.000	
max	15.000	56.230	365.000	356672.540	53.000	0.157	0.227	12.000	6.000	4.000	

In [4]:

```
# Spot-check: pick one patient and verify visit_frequency manually
sample_pid = df["patient_id"].iloc[0]
expected_freq = df[df["patient_id"] == sample_pid]["visit_id"].nunique()
actual_freq = df[df["patient_id"] == sample_pid]["visit_frequency"].iloc[0]
print(f"Spot-check patient_id={sample_pid}: expected visit_frequency={expected_freq}, got={actual_freq}")
assert expected_freq == actual_freq, "visit_frequency mismatch!"
print("✓ Spot-check passed\n")

# Save
output_path = os.path.join(os.path.dirname(os.path.dirname(os.getcwd())), "Database", "model_table.csv")
df.to_csv(output_path, index=False)
print(f"Saved to {output_path}")
print(f"Final columns ({len(df.columns)}): {list(df.columns)}")
```

```
Spot-check patient_id=2964: expected visit_frequency=4, got=4
✓ Spot-check passed

Saved to /Users/ramsundar/Documents/IITM-Graded-Projects/Capstone/Database/model_table.csv
Final columns (33): ['patient_id', 'age', 'gender', 'city', 'insurance_provider', 'chronic_flag', 'registration_date', 'visit_id', 'visit_date', 'department', 'visit_type', 'le
gth_of_stay_hours', 'risk_score', 'doctor_id', 'bill_id', 'billed_amount', 'approved_amount', 'claim_status', 'payment_days', 'billing_date', 'visit_frequency', 'avg_length_of_
tay', 'days_since_registration', 'total_billed', 'avg_payment_days', 'provider_rejection_rate', 'doctor_rejection_rate', 'visit_month', 'visit_day_of_week', 'visit_quarter', '
_weekend', 'days_between_reg_and_visit', 'billing_delay_days']
```